
INF1350

Sistemas Reativos

Interrupções

Prof.: Adriano Branco

abranco@inf.puc-rio.br

Slides baseado em aulas anteriores da Prof. Noemi Rodriguez e com contribuição do Prof. Francisco Sant'Anna e do Prof. Adriano Branco

o que são interrupções

- interrupção do programa em execução
 - Síncronas
 - Assíncronas
 - forma do hardware chamar atenção!

o que são interrupções

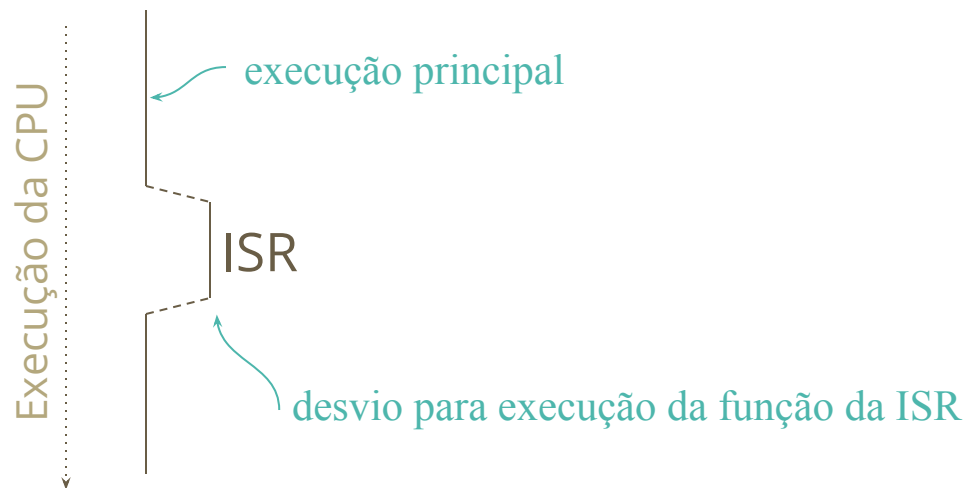
- interrupção do programa em execução
 - Síncronas
 - Assíncronas
 - forma do hardware chamar atenção!

similaridade com uma callback gerada pelo hardware!

- mas o programa em execução é interrompido!

tratamento de interrupções

- hardware desvia controle para tratador
 - configuração específica de cada CPU



interrupções - utilidade

- alternativa ao polling: avisos sobre operações completas
- tratamento de falhas (page faults, etc)
- preempção de processos e threads
- . . .
- chamadas ao sistema operacional
- . . .

interrupções - tipos

- transmissão/recebimento de dados
 - chegada de dados
 - erro na transmissão
 - ...
- erros de execução
 - segfault (acesso a endereço inválido)
 - código de máquina inválido
- syscall

interrupções - cuidados

- troca de contexto
 - registradores
 - pilha de execução
 - ...
- tratamento bloqueia outros acontecimentos
 - interrupção da interrupção?
 - tratamento tem que ser rápido

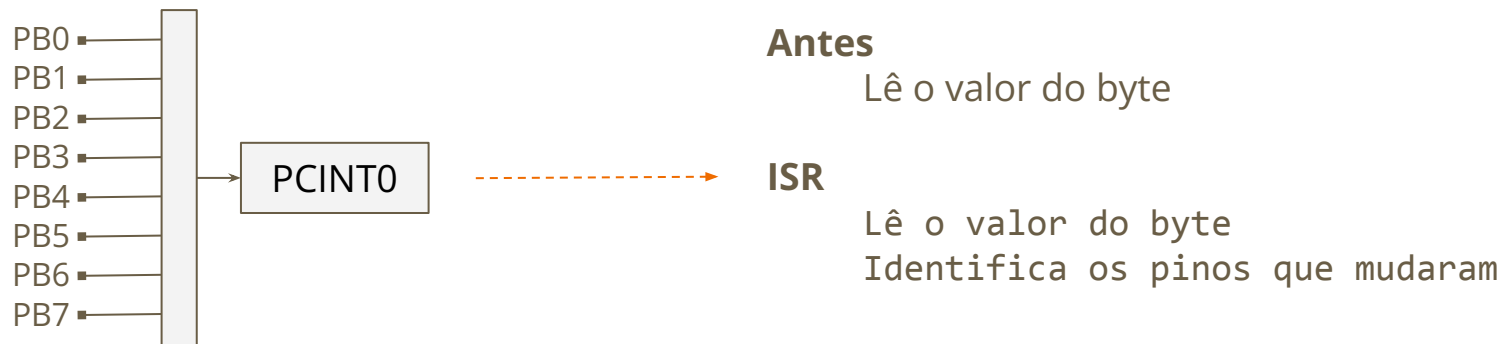
interrupções - avr

- suporte do compilador
 - registradores são automaticamente salvos e restaurados
- tratadores de novas interrupções executados apenas depois que tratador em execução terminar
 - em ordem de prioridades
- tratadores por pinos, conjuntos de pinos ou periféricos
 - `ISR (INT0_vect)` interrupt for pin D2
 - `ISR (PCINT0_vect)` pin change interrupt for D8 to D13
 - `ISR (PCINT2_vect)` pin change interrupt for D0 to D7
 - `ISR (ADC_vect)` ADC conversion complete

avr interrupt vector ATmega328

1	Reset	
2	External Interrupt Request 0 (pin D2)	(INT0_vect)
3	External Interrupt Request 1 (pin D3)	(INT1_vect)
4	Pin Change Interrupt Request 0 (pins D8 to D13)	(PCINT0_vect)
5	Pin Change Interrupt Request 1 (pins A0 to A5)	(PCINT1_vect)
6	Pin Change Interrupt Request 2 (pins D0 to D7)	(PCINT2_vect)
7	Watchdog Time-out Interrupt	(WDT_vect)
8	Timer/Counter2 Compare Match A	(TIMER2_COMPA_vect)
9	Timer/Counter2 Compare Match B	(TIMER2_COMPB_vect)
10	Timer/Counter2 Overflow	(TIMER2_OVF_vect)
11	Timer/Counter1 Capture Event	(TIMER1_CAPT_vect)
12	Timer/Counter1 Compare Match A	(TIMER1_COMPA_vect)
13	Timer/Counter1 Compare Match B	(TIMER1_COMPB_vect)
14	Timer/Counter1 Overflow	(TIMER1_OVF_vect)
15	Timer/Counter0 Compare Match A	(TIMER0_COMPA_vect)
16	Timer/Counter0 Compare Match B	(TIMER0_COMPB_vect)
17	Timer/Counter0 Overflow	(TIMER0_OVF_vect)
18	SPI Serial Transfer Complete	(SPI_STC_vect)
19	USART Rx Complete	(USART_RX_vect)
20	USART, Data Register Empty	(USART_UDRE_vect)
21	USART, Tx Complete	(USART_TX_vect)
22	ADC Conversion Complete	(ADC_vect)
23	EEPROM Ready	(EE_READY_vect)
24	Analog Comparator	(ANALOG_COMP_vect)
25	2-wire Serial Interface (I2C)	(TWI_vect)
26	Store Program Memory Ready	(SPM_READY_vect)

Interrupção de Pino (INT) vs. de Barramento (PCINT)



interrupções – como tratar

```
volatile int buttonChanged = 0;
```

```
void pciSetup (byte pin) {  
    *digitalPinToPCMSK(pin) |= bit (digitalPinToPCMSKbit(pin)); // enable pin  
    PCIFR |= bit (digitalPinToPCICRbit(pin)); // clear any outstanding interrupt  
    PCICR |= bit (digitalPinToPCICRbit(pin)); // enable interrupt for the group  
}
```

```
ISR (PCINT1_vect) { // handle pin change interrupt for A0 to A5 here  
    buttonChanged=1;  
}  
...
```

O `volatile` força o acesso direto ao endereço de memória, impedindo que o compilador otimize e remova leituras ou escritas em variáveis e registradores de I/O.

interrupções – como tratar

```
...  
void setup() {  
  pinMode(LED1, OUTPUT);    pinMode(LED2, OUTPUT);  
  digitalWrite(LED1,HIGH);  digitalWrite(LED2,HIGH);  
  pinMode (KEY1, INPUT_PULLUP); pinMode (KEY2, INPUT_PULLUP);  
  pciSetup(KEY1);           pciSetup(KEY2);  
  Serial.begin(9600);  
}  
  
void loop() {  
  Serial.print("!"); delay(10);  
  if (buttonChanged) {  
    digitalWrite(LED1,digitalRead(KEY1));  
    digitalWrite(LED2,digitalRead(KEY2));  
    buttonChanged = 0;  
  }  
}
```

interrupções – quando usar

- tempo de resposta
- compatibilidade com tarefas de usuário de longa duração
- Dificuldades:
 - problemas de concorrência: não podemos usar locks comuns
 - `volatile` resolve a visibilidade, não a atomicidade
 - tratamento tem que ser rápido

volatile não basta

- `volatile` garante que a leitura acontece de verdade. Não garante que ela seja indivisível.
- num AVR de 8 bits, um `int` de 16 bits é lido em duas instruções, e a interrupção pode cair no meio.
- `counter++` são três passos: ler, somar, escrever.
- o mesmo vale sem depender do tamanho: testar `buttonChanged` e depois zerar são duas operações, e uma interrupção entre elas apaga um evento que acabou de chegar.

atomicidade

- mutex não serve: a ISR não pode esperar por quem ela interrompeu
- a proteção fica no programa principal, que desliga as interrupções por um instante
- a ISR já roda com as interrupções desabilitadas

```
#include <util/atomic.h>
```

```
// sem proteção:
```

```
if (counter > 50) {  
    ...;  
    counter = 0;  
}
```

```
#include <util/atomic.h>
```

```
// com proteção:
```

```
int c;  
ATOMIC_BLOCK(ATOMIC_RESTORESTATE) { c = counter; }  
if (c > 50) {  
    ATOMIC_BLOCK(ATOMIC_RESTORESTATE) { counter -= c; }  
    ...  
}
```

Dentro do bloco atômico as interrupções ficam desligadas, e isso aumenta a latência. Mantenha o bloco curto, ou você perde o tempo de resposta que motivou usar interrupção.

interrupções – outro uso

- dispositivos podem ser colocados em "modo sleep" para economizar energia
 - no arduíno, economia é um pouco artificial
 - conceito é importante em vários contextos

interrupções – outro uso

```
#include <avr/sleep.h>
#include <avr/power.h>

...
void enterSleep(void){
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    sleep_enable();
    sleep_mode(); /* The program will continue from here. */
    /* First thing to do is disable sleep. */
    sleep_disable();
}
```

PWR_DOWN para quase tudo, inclusive os timers; **IDLE** para só o núcleo. Quanto mais o modo desliga, mais economiza e menos coisas conseguem acordar o processador.

escalonador de tarefas

- Até aqui vimos três formas de reagir: amostragem no loop, callbacks sobre amostragem, interrupção
- falta organizar várias reações convivendo: piscar, ler chaves, medir tempo, tudo junto
- ideia: a aplicação vira um conjunto de tarefas curtas, e alguém decide quando cada uma roda
 - a interrupção não executa a tarefa: coloca na fila e retorna
 - o loop tira da fila e executa, uma de cada vez, até esvaziar
- temporizador virtual: um único timer de hardware atende várias tarefas, cada uma com seu prazo
- continua valendo a hipótese síncrona: uma tarefa nunca interrompe a outra

Base de tempo de 16 ms, pegando carona na interrupção que o Arduino já usa para o millis. Não ocupa Timer1 nem Timer2, então tone, Servo e o display do shield continuam disponíveis.

tarefas e API

Uma tarefa é uma função curta que recebe um byte e retorna:

```
void minhaTarefa(uint8_t arg);
```

Toda tarefa precisa ser registrada, e é no registro que se declara o tipo:

```
registerTask(app, keyTask);
```

```
registerTask(vtimer, blinkTask);
```

- app: entra na fila quando alguém a posta, inclusive de dentro de uma ISR
- vtimer: entra na fila sozinha, quando o prazo do temporizador vence

```
void initScheduler(void);
```

```
int8_t registerTask(taskType_t tipo, task_ptr_t fx);
```

```
int8_t postTask(taskType_t tipo, task_ptr_t fx, uint8_t arg);
```

```
int8_t startVTimer(task_ptr_t fx, uint16_t ms, uint8_t arg);
```

```
int8_t stopVTimer(task_ptr_t fx);
```

```
void procTasks(void);
```

- registrar antes de postar a tarefa ou de iniciar o seu temporizador
- o temporizador dispara uma vez; para repetir, a própria tarefa reinicia o temporizador
- toda tarefa precisa retornar, senão nenhuma outra executa

exemplo: registro e interrupção

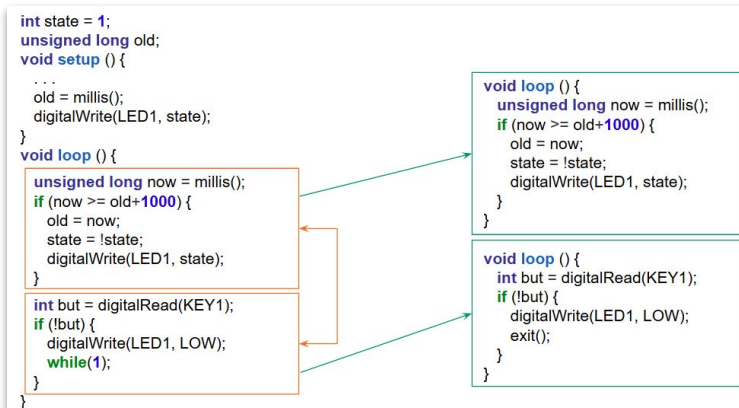
```
void setup() {  
    ...  
    initScheduler();  
    registerTask(vtimer, blinkTask);  
    registerTask(app, keyTask);  
    pciSetup(KEY1);  
    startVTimer(blinkTask, 1000, 0);  
}  
ISR(PCINT1_vect) { // não decide nada  
    postTask(app, keyTask, digitalRead(KEY1) == LOW);  
}  
void loop() {  
    procTasks(); // executa o que estiver na fila  
    enterSleep(); // dorme em IDLE até o próximo evento  
}
```

- a ISR traduz o nível elétrico em valor lógico e posta a tarefa, sem tocar no LED
- fila vazia, nada a fazer: dormir é melhor que girar em falso

exemplo: as tarefas

```
void blinkTask(uint8_t arg) {  
    ledAceso = !ledAceso;  
    digitalWrite(LED1, ledAceso ? LOW : HIGH);  
    startVTimer(blinkTask, 1000, 0); // reinicia o temporizador  
}
```

```
void keyTask(uint8_t pressionada) {  
    if (!pressionada) return; // ignora o soltar  
    stopVTimer(blinkTask);  
    digitalWrite(LED1, LOW);  
}
```



- o mesmo pisca e para das aulas anteriores, agora em duas tarefas independentes
- keyTask retorna: sem `while(1)`, o escalonador continua vivo
- código completo em: <https://www.inf.puc-rio.br/~abranco/sr-23.2/code/arduino/blinkScheduler/>

Exercício (não é para entregar no EAD)

Baixar a aplicação mostrada, led aceso quando a chave é pressionada:

<https://www.inf.puc-rio.br/~abranco/sr-23.2/code/arduino/interruptaochave/>

1. Transformar em uma aplicação que liga e desliga o LED 1 a cada pressionamento da chave 1, sem se preocupar com debounce. Dois pontos a resolver antes de acusar o ruído de contato:
 - o o PCINT dispara ao pressionar e ao soltar
 - o a ISR só avisa que algo mudou no grupo, não diz qual chave
2. Usar `millis` para tratar o debounce, contando o tempo desde a última interrupção tratada.
3. Substituir o `millis` por contagem via interrupção de timer. Ver o exemplo em: <https://www.inf.puc-rio.br/~abranco/sr-23.2/code/arduino/timer/>. Cada estouro vale cerca de 16 ms, não 1 ms. Leve isso em conta ao dimensionar o tempo de debounce.
4. Aplicar `ATOMIC_BLOCK` no acesso ao contador e ao `buttonChanged`, e explicar por que cada um precisa.
5. Voltando à aplicação original, chamar `enterSleep()` dentro do loop, acordando pelas interrupções das chaves.